

Lab 4 — Versioning, Feature Store & Lineage

Track A (tabular fraud-detection) · Week 4 · DS5619 Machine Learning Systems Operations

Setup

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python generate_for_student.py --student-id <your roll number or institute email>
```

This overwrites `data/v1/transactions.csv` and `data/v2/transactions.csv` with records generated deterministically from your student ID — same shape as everyone else's, different actual values.

Record your `--student-id` value in `NOTES.md`. The grader re-runs `generate_for_student.py` with the ID you recorded and diffs the result against what you committed.

Files

- `src/mini_feature_store.py` — implement the four `# TODO` functions.
- `src/run_pipeline.py` — complete driver script, runs your functions against `data/v1/` then `data/v2/`. Don't edit.
- `data/v1/transactions.csv`, `data/v2/transactions.csv` — your two schema revisions of the same feed, generated above (don't hand-edit).

Background

`data/v1/transactions.csv` and `data/v2/transactions.csv` are two revisions of the same upstream transaction feed. Between v1 and v2, the upstream team made a **breaking schema change**: `country` was renamed to `country_code`, `amount` (float) became `amount_minor_units` (integer cents), and a new `device_fingerprint` field was added. This is deliberately the same kind of change Week 2/3 warned you real upstream systems make without notice.

A correct versioning + feature store setup should handle this without anyone touching history: v1 stays exactly as it was recorded, and v2 becomes a new, separate version — of both the raw data AND the feature group built from it.

Your task

Part 1-4 — `src/mini_feature_store.py` (four functions marked `# TODO`, each has a full docstring spec, ~45 min total)

- `snapshot_raw_version(input_path, registry_dir)` — content-hash-based, idempotent raw data versioning.

- `build_features(rows)` — per-`card_id` aggregate features; must correctly handle both the v1 and v2 schemas (detect which you're given, normalize before aggregating).
- `register_feature_group(name, feature_rows, source_version_id, registry_dir, transform_version)` — writes a new feature group version + its lineage manifest; must never overwrite a previous version.
- `get_lineage(name, fg_version_id, registry_dir)` — reads a feature group's manifest and its source raw version's manifest, returns the combined chain.

```
python src/run_pipeline.py
```

This runs your four functions against v1, then v2, checks that re-snapshotting v1 is idempotent, and writes `lineage_report.json` at the repo root (`src/run_pipeline.py` is complete, don't edit it).

Self-check

```
pytest tests/ -q
```

This is a self-check, not the grader.

Deliverables (what you commit)

- `src/mini_feature_store.py`, completed.
- The `.feature_store/` directory your pipeline run produced (it's small — JSON manifests only, no raw data copies of meaningful size).
- `lineage_report.json`.
- A short `NOTES.md`: the `--student-id` value you used (required — see above), plus what's different between the v1 and v2 feature group's `manifest.json` (look at both), and why does `build_features` need to treat `amount_minor_units` differently from `amount` for the aggregates to be comparable across versions?

Grading checklist

- ☐ `data/` matches what `generate_for_student.py --student-id <NOTES.md value>` actually produces.
- ☐ `snapshot_raw_version` is genuinely idempotent (same content → same version id, verified against a held-out file, not just the provided one).
- ☐ `build_features` produces correct aggregates for both schemas, and the v2 amounts are correctly converted from minor units before aggregating.
- ☐ `register_feature_group` never overwrites an existing version — running the pipeline twice results in v1 and v2 (not just v1 again).
- ☐ `get_lineage` correctly resolves a feature group back to its raw source manifest.
- ☐ `NOTES.md` shows you actually compared the v1 and v2 manifests.
- ☐ Meaningful commit history and a working README.

Submission

```
git add -A  
git commit -m "Week 4: mini feature store + lineage"  
git tag week04-submit  
git push origin main --tags
```

Source: this lab operationalizes the Data Versioning, Feature Store (FTI architecture, feature groups), and Data Lineage content from the Week 4 lecture deck.